

SQL Practical Exam

Complete Study Guide

All 13 Practicals · Step-by-Step · With Output

Works on Ubuntu & Windows · MySQL

Quick Reference — All Practicals

#	Topic	Key Commands
1	Design & Implement DB	CREATE DATABASE, CREATE TABLE, INSERT INTO, SELECT
2	PK & FK Constraints	PRIMARY KEY, FOREIGN KEY REFERENCES
3	Relational Operators	WHERE, =, !=, >, <, BETWEEN, LIKE, IN
4	Set Operators	UNION, workarounds for INTERSECT / MINUS
5	Views	CREATE VIEW, SELECT, UPDATE, DROP VIEW
6	Triggers	CREATE TRIGGER, AFTER UPDATE, OLD/NEW
7	Stored Procedures	CREATE PROCEDURE, CALL, IF/ELSEIF
8	Functions	CREATE FUNCTION, RETURNS, RETURN
9	Constraints	PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE
10	Aggregate Functions	COUNT, SUM, AVG, MAX, MIN
11	Nested Queries	Subquery in WHERE, IN (SELECT ...)

12	Indexes	CREATE INDEX, SHOW INDEX, DROP INDEX
13	Sorting & Grouping	ORDER BY, GROUP BY, HAVING

Q1

Design & Implement a Student Database

Create a database, define tables using DDL, and insert data using DML.

STEP 1 — OPEN MYSQL

NOTE: Ubuntu Terminal: type `mysql -u root -p` then enter your password. Windows: open MySQL Command Line Client from the Start Menu.

STEP 2 — CREATE DATABASE & TABLES (DDL)

```
-- Create the database

CREATE DATABASE StudentDB;

USE StudentDB;

-- Create Student table

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT,
    Department VARCHAR(30),
    Marks DECIMAL(5,2)
);
```

STEP 3 — INSERT DATA (DML)

```
INSERT INTO Student VALUES
(1, 'Aarav Shah', 20, 'Computer', 88.5),
(2, 'Priya Patel', 21, 'Maths', 75.0),
(3, 'Rohit Desai', 22, 'Computer', 91.0),
(4, 'Sneha Kulkarni', 20, 'Science', 65.5),
(5, 'Dev Joshi', 23, 'Maths', 79.0);

-- View all data

SELECT * FROM Student;
```

EXPECTED OUTPUT

```
+-----+-----+-----+-----+
| StudentID | Name | Age | Department | Marks |
+-----+-----+-----+-----+
| 1 | Aarav Shah | 20 | Computer | 88.50 |
| 2 | Priya Patel | 21 | Maths | 75.00 |
| 3 | Rohit Desai | 22 | Computer | 91.00 |
| 4 | Sneha Kulkarni | 20 | Science | 65.50 |
| 5 | Dev Joshi | 23 | Maths | 79.00 |
+-----+-----+-----+-----+

5 rows in set
```

Q2**Primary Key & Foreign Key Constraints**

Create two related tables — Course and Enrollment — linked by a Foreign Key.

STEP 1 — CREATE PARENT TABLE (COURSE)

```
CREATE TABLE Course (  
  CourseID INT PRIMARY KEY,  
  CourseName VARCHAR(50) NOT NULL,  
  Credits INT  
);
```

STEP 2 — CREATE CHILD TABLE WITH FOREIGN KEY

```
CREATE TABLE Enrollment (  
  EnrollID INT PRIMARY KEY,  
  StudentID INT,  
  CourseID INT,  
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

STEP 3 — INSERT DATA & TEST FK

```
INSERT INTO Course VALUES (101, 'Database Systems', 4);  
INSERT INTO Course VALUES (102, 'Operating Systems', 3);  
  
INSERT INTO Enrollment VALUES (1, 1, 101);  
INSERT INTO Enrollment VALUES (2, 3, 101);  
INSERT INTO Enrollment VALUES (3, 2, 102);  
  
-- This will FAIL – StudentID 99 does not exist  
INSERT INTO Enrollment VALUES (4, 99, 101);
```

FK ERROR OUTPUT (CORRECT BEHAVIOUR!)

```
ERROR 1452 (23000): Cannot add or update a child row:  
a foreign key constraint fails  
This error is CORRECT – FK constraint is working!
```

KEY POINT

NOTE: Parent table must have the record **FIRST** before the child table can reference it. **OLD** values = before change, **NEW** values = after change.

Q3

SQL Queries with Relational Operators

Use =, !=, <, >, BETWEEN, LIKE, IN on the Student table.

RELATIONAL OPERATOR QUERIES

```
-- Equal to
SELECT * FROM Student WHERE Department = 'Computer';

-- Greater than
SELECT * FROM Student WHERE Marks > 80;

-- BETWEEN (inclusive range)
SELECT * FROM Student WHERE Marks BETWEEN 70 AND 90;

-- LIKE (% means anything)
SELECT * FROM Student WHERE Name LIKE 'A%';

-- IN (list of values)
SELECT * FROM Student WHERE Department IN ('Computer', 'Maths');

-- NOT EQUAL
SELECT * FROM Student WHERE Department != 'Science';
```

OUTPUT: WHERE MARKS > 80

```
+-----+-----+-----+-----+-----+
| StudentID | Name | Age | Department | Marks |
+-----+-----+-----+-----+-----+
| 1 | Aarav Shah | 20 | Computer | 88.50 |
| 3 | Rohit Desai | 22 | Computer | 91.00 |
+-----+-----+-----+-----+-----+
2 rows in set
```

Q4

Set Operators: UNION, INTERSECT, MINUS

Combine query results. MySQL needs workarounds for INTERSECT and MINUS.

MYSQL NOTE

NOTE: MySQL does NOT support INTERSECT or MINUS directly. Use the subquery workarounds shown below. PostgreSQL supports all three natively.

UNION — ALL STUDENTS FROM BOTH TABLES (NO DUPLICATES)

```
SELECT StudentID, Name FROM Student
UNION
SELECT StudentID, Name FROM ScienceStudents;
```

UNION OUTPUT

```
+-----+-----+
| StudentID | Name |
```

```
+-----+-----+
| 1 | Aarav Shah |
| 2 | Priya Patel |
| 3 | Rohit Desai |
| 4 | Sneha Kulkarni | <-- once only
| 5 | Dev Joshi |
| 6 | Meera Singh |
+-----+-----+
```

INTERSECT WORKAROUND (MYSQL)

```
-- Students present in BOTH tables

SELECT StudentID, Name FROM Student
WHERE StudentID IN (SELECT StudentID FROM ScienceStudents);
```

MINUS WORKAROUND (MYSQL)

```
-- Students in Student but NOT in ScienceStudents

SELECT StudentID, Name FROM Student
WHERE StudentID NOT IN (SELECT StudentID FROM ScienceStudents);
```

Q5

DDL & DML Statements on Views

Create a VIEW (virtual table), query it, update through it, and drop it.

STEP 1 — CREATE A VIEW

```
-- View shows only Computer dept students

CREATE VIEW ComputerStudents AS

SELECT StudentID, Name, Marks

FROM Student

WHERE Department = 'Computer';
```

STEP 2 — QUERY THE VIEW

```
SELECT * FROM ComputerStudents;
```

VIEW OUTPUT

```
+-----+-----+-----+
| StudentID | Name | Marks |
+-----+-----+-----+
| 1 | Aarav Shah | 88.50 |
| 3 | Rohit Desai | 91.00 |
+-----+-----+-----+
```

STEP 3 — UPDATE THROUGH VIEW & DROP

```
-- Update marks through the view

UPDATE ComputerStudents SET Marks = 95.0 WHERE StudentID = 3;

-- Drop the view (does NOT delete table data)

DROP VIEW ComputerStudents;
```

KEY POINT

NOTE: A VIEW is a saved SELECT query. Updating through a view updates the REAL table. Dropping a view only removes the definition — table data is safe.

Q6

Database Triggers

Auto-log changes to marks whenever a student record is updated.

STEP 1 — CREATE AUDIT LOG TABLE

```
CREATE TABLE MarksAudit (

AuditID INT AUTO_INCREMENT PRIMARY KEY,

StudentID INT,

OldMarks DECIMAL(5,2),

NewMarks DECIMAL(5,2),

ChangedOn DATETIME DEFAULT NOW()

);
```

STEP 2 — CREATE THE TRIGGER

```

DELIMITER //

CREATE TRIGGER AfterMarksUpdate
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
IF OLD.Marks != NEW.Marks THEN
INSERT INTO MarksAudit (StudentID, OldMarks, NewMarks)
VALUES (NEW.StudentID, OLD.Marks, NEW.Marks);
END IF;
END//

DELIMITER ;

```

STEP 3 — FIRE THE TRIGGER & CHECK LOG

```

UPDATE Student SET Marks = 92.0 WHERE StudentID = 1;

SELECT * FROM MarksAudit;

```

AUDIT LOG OUTPUT

```

+-----+-----+-----+-----+-----+
| AuditID | StudentID | OldMarks | NewMarks | ChangedOn |
+-----+-----+-----+-----+-----+
| 1 | 1 | 88.50 | 92.00 | 2025-01-01 10:30:00 |
+-----+-----+-----+-----+-----+

```

TRIGGER TYPES

NOTE: BEFORE/AFTER = when it fires. INSERT/UPDATE/DELETE = which operation. OLD.column = value before change. NEW.column = value after change.

Q7

PL/SQL Stored Procedures

A procedure takes a StudentID input, calculates grade, and prints it.

STEP 1 — CREATE THE PROCEDURE

```
DELIMITER //
```

```
CREATE PROCEDURE GetStudentGrade(IN sid INT)
```

```
BEGIN
```

```
DECLARE sname VARCHAR(50);
```

```
DECLARE smarks DECIMAL(5,2);
```

```
DECLARE grade VARCHAR(2);
```

```
SELECT Name, Marks INTO sname, smarks
```

```
FROM Student WHERE StudentID = sid;
```

```
IF smarks >= 90 THEN SET grade = 'A';
```

```
ELSEIF smarks >= 75 THEN SET grade = 'B';
```

```
ELSEIF smarks >= 60 THEN SET grade = 'C';
```

```
ELSE SET grade = 'F';
```

```
END IF;
```

```
SELECT sname AS 'Student', smarks AS 'Marks', grade AS 'Grade';
```

```
END//
```

```
DELIMITER ;
```

STEP 2 — CALL THE PROCEDURE

```
CALL GetStudentGrade(3); -- Rohit Desai, 95 marks
```

```
CALL GetStudentGrade(4); -- Sneha Kulkarni, 65.5 marks
```

OUTPUT

```
+-----+-----+-----+
| Student | Marks | Grade |
+-----+-----+-----+
| Rohit Desai | 95.00 | A |
+-----+-----+-----+

+-----+-----+-----+
| Student | Marks | Grade |
+-----+-----+-----+
| Sneha Kulkarni | 65.50 | C |
+-----+-----+-----+
```

Q8

PL/SQL Functions

A function returns a grade value — can be used directly inside SELECT.

PROCEDURE VS FUNCTION

NOTE: PROCEDURE performs actions (print, insert, update) — called with CALL. FUNCTION calculates and RETURNS a value — used directly inside SELECT.

STEP 1 — CREATE THE FUNCTION

```
DELIMITER //
```

```
CREATE FUNCTION CalculateGrade(marks DECIMAL(5,2))
```

```
RETURNS VARCHAR(2)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
DECLARE grade VARCHAR(2);
```

```
IF marks >= 90 THEN SET grade = 'A';
```

```
ELSEIF marks >= 75 THEN SET grade = 'B';
```

```
ELSEIF marks >= 60 THEN SET grade = 'C';
```

```
ELSE SET grade = 'F';
```

```
END IF;
```

```
RETURN grade;
```

```
END//
```

```
DELIMITER ;
```

STEP 2 — USE FUNCTION IN SELECT

```
SELECT Name, Marks, CalculateGrade(Marks) AS Grade
```

```
FROM Student;
```

OUTPUT

```
+-----+-----+-----+
| Name | Marks | Grade |
+-----+-----+-----+
| Aarav Shah | 92.00 | A |
| Priya Patel | 75.00 | B |
| Rohit Desai | 95.00 | A |
| Sneha Kulkarni | 65.50 | C |
| Dev Joshi | 79.00 | B |
+-----+-----+-----+
```

Q9

Constraints: PK, FK, NOT NULL, UNIQUE

Create a table demonstrating all four constraint types with test cases.

CREATE TABLE WITH ALL CONSTRAINTS

```
CREATE TABLE CollegeStudent (  
RollNo INT PRIMARY KEY, -- Unique, Not Null  
Email VARCHAR(100) UNIQUE NOT NULL, -- No duplicates allowed  
FullName VARCHAR(50) NOT NULL, -- Cannot be empty/null  
DeptID INT,  
FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

TEST EACH CONSTRAINT (THESE WILL FAIL CORRECTLY)

```
-- WORKS fine  
INSERT INTO CollegeStudent VALUES (101,'aarav@email.com','Aarav Shah',1);  
  
-- FAILS: Duplicate RollNo (PRIMARY KEY violation)  
INSERT INTO CollegeStudent VALUES (101,'new@email.com','Someone',1);  
  
-- FAILS: NULL FullName (NOT NULL violation)  
INSERT INTO CollegeStudent VALUES (102,'b@email.com', NULL, 1);  
  
-- FAILS: Duplicate email (UNIQUE violation)  
INSERT INTO CollegeStudent VALUES (103,'aarav@email.com','Priya',1);
```

CONSTRAINT ERROR MESSAGES

```
ERROR 1062: Duplicate entry '101' for key 'PRIMARY'  
ERROR 1048: Column 'FullName' cannot be null  
ERROR 1062: Duplicate entry 'aarav@email.com' for key 'Email'  
  
These errors are CORRECT – constraints are working!
```

**Q1
0**

Aggregate Functions

Use COUNT, SUM, AVG, MAX, MIN to summarise Student table data.

ALL AGGREGATE FUNCTIONS

```
-- Count of students  
SELECT COUNT(*) AS TotalStudents FROM Student;  
  
-- Sum of all marks  
SELECT SUM(Marks) AS TotalMarks FROM Student;  
  
-- Average marks  
SELECT AVG(Marks) AS AverageMarks FROM Student;  
  
-- Highest marks  
SELECT MAX(Marks) AS HighestMarks FROM Student;
```

```
-- Lowest marks

SELECT MIN(Marks) AS LowestMarks FROM Student;

-- All together in one query

SELECT COUNT(*) AS Total, SUM(Marks) AS Sum,
AVG(Marks) AS Average, MAX(Marks) AS Highest,
MIN(Marks) AS Lowest
FROM Student;
```

COMBINED OUTPUT

```
+-----+-----+-----+-----+-----+
| Total | Sum | Average | Highest | Lowest |
+-----+-----+-----+-----+-----+
| 5 | 426.50 | 85.3000 | 95.00 | 65.50 |
+-----+-----+-----+-----+-----+
```

Q1 1

Nested Queries & Subqueries

Use a query inside another query to get filtered results.

SUBQUERY EXAMPLES

```
-- Students with above-average marks
SELECT Name, Marks FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);

-- Student with the highest marks
SELECT Name, Marks FROM Student
WHERE Marks = (SELECT MAX(Marks) FROM Student);

-- Students enrolled in 'Database Systems' (nested subquery)
SELECT Name FROM Student
WHERE StudentID IN (
  SELECT StudentID FROM Enrollment
  WHERE CourseID = (
    SELECT CourseID FROM Course
    WHERE CourseName = 'Database Systems'
  )
);
```

ABOVE-AVERAGE OUTPUT (AVG = 85.3)

```
+-----+-----+
| Name | Marks |
+-----+-----+
| Aarav Shah | 92.00 |
| Rohit Desai | 95.00 |
+-----+-----+
```

HOW TO READ A SUBQUERY

NOTE: Read INNER query first, then OUTER query. Inner = 'find average marks' = 85.3 Outer = 'find students where marks > 85.3'

Q1 2

Index Operations

Create indexes to speed up searches. Show, create, and drop indexes.

CREATING AND USING INDEXES

```
-- Simple index on Department column
CREATE INDEX idx_department ON Student(Department);

-- Unique index (like UNIQUE constraint)
CREATE UNIQUE INDEX idx_email ON CollegeStudent(Email);

-- Composite index (two columns together)
```

```
CREATE INDEX idx_dept_marks ON Student(Department, Marks);

-- View all indexes on a table

SHOW INDEX FROM Student;

-- Drop an index

DROP INDEX idx_department ON Student;
```

SHOW INDEX OUTPUT

```
+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Column_name |
+-----+-----+-----+-----+
| student | 0 | PRIMARY | StudentID |
| student | 1 | idx_department | Department |
| student | 1 | idx_dept_marks | Department |
| student | 1 | idx_dept_marks | Marks |
+-----+-----+-----+-----+
```

WHEN TO USE INDEXES

NOTE: Use INDEX on columns in WHERE, JOIN, ORDER BY. Non-unique index = allows duplicates. Unique index = no duplicates. Indexes speed up SELECT but slow down INSERT/UPDATE slightly.

Q1
3

Sorting & Grouping: ORDER BY, GROUP BY, HAVING

Sort results and group rows to get per-group statistics.

ORDER BY — SORT RESULTS

```
-- Highest to lowest marks
SELECT Name, Marks FROM Student
ORDER BY Marks DESC;

-- Alphabetical by name
SELECT Name, Department FROM Student
ORDER BY Name ASC;
```

ORDER BY DESC OUTPUT

```
+-----+-----+
| Name | Marks |
+-----+-----+
| Rohit Desai | 95.00 |
| Aarav Shah | 92.00 |
| Dev Joshi | 79.00 |
| Priya Patel | 75.00 |
| Sneha Kulkarni | 65.50 |
+-----+-----+
```

GROUP BY — GROUP ROWS

```
SELECT Department,
COUNT(*) AS StudentCount,
AVG(Marks) AS AvgMarks
FROM Student
GROUP BY Department;
```

GROUP BY OUTPUT

```
+-----+-----+-----+
| Department | StudentCount | AvgMarks |
+-----+-----+-----+
| Computer | 2 | 93.5000 |
| Maths | 2 | 77.0000 |
| Science | 1 | 65.5000 |
+-----+-----+-----+
```

HAVING — FILTER GROUPS

```
-- Only departments with average marks above 75
SELECT Department, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY Department
HAVING AvgMarks > 75;
```

```
HAVING AVG(Marks) > 75;
```

HAVING OUTPUT

```
+-----+-----+
```

```
| Department | AvgMarks |
```

```
+-----+-----+
```

```
| Computer | 93.5000 |
```

```
| Maths | 77.0000 |
```

```
+-----+-----+
```

```
(Science excluded – avg 65.5 is not > 75)
```

WHERE VS HAVING

NOTE: WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER grouping. Rule: filter on aggregate (COUNT, AVG, SUM) -> use HAVING. Filter on plain column -> use WHERE.